

# Spotter — Machine Learning Engineer Assessment

Freight rate prediction — merged solution: CatBoost, rate-per-mile target, two-model December handling

---

## 1. Summary

The final model is a CatBoostRegressor trained on  $\log(\text{posted\_rate} / \text{distance})$  — i.e. it predicts rate per mile, which is then multiplied back by distance to recover the dollar prediction. On a time-based holdout (train Jan–Sep 2025, validate on October, which the model never sees during training) it scores MAE \$114.67, RMSE \$649.41, median absolute error \$41.66, and MAPE 5.48%. Two versions are trained: a full-feature model (includes market\_index/quote\_signal) used for validation\_predictions.csv, and a reduced-feature model (excludes them, since december\_chart\_inputs.csv doesn't provide them) used only for the December chart.

- Target:  $\log(\text{rate per mile})$ , not  $\log(\text{rate})$  — divides out the dominant distance effect before the model starts, which measurably improves accuracy over modeling the raw rate.
- Categoricals: pickup, delivery, equipment, route (pickup+delivery), route\_equipment — handled natively by CatBoost, no one-hot encoding needed.
- Weight: only the corrected weight\_abs plus a weight\_missing flag are used as features. The raw signed value is dropped — see Section 3.
- Validation: time-based holdout is the headline number; a random-split comparison and a new unseen-city experiment (Section 5) are run alongside it to pressure-test the validation methodology itself, not just the model.
- December: predicted rate for the fixed Lexington → Fort Wayne lane ranges \$811–\$835 across the 31 days of December, with a clean weekly cycle.

## 2. What changed, and why (merge notes)

This report merges two earlier draft solutions to the same assessment. Rather than pick one wholesale, each design decision was evaluated on its own merits and, where the evidence supported it, adopted, dropped, or modified:

- Adopted: modeling  $\log(\text{rate per mile})$  instead of  $\log(\text{rate})$ . This was the single biggest accuracy win in either draft — it's the reason this model beats a plain  $\log(\text{rate})$  gradient-boosting model on MAE and MAPE.
- Adopted, then verified: route/route\_equipment categoricals. These were kept, but Section 6's feature importance shows they contribute far less than expected once pickup/delivery/geo\_distance are already in the model (route ranks near the bottom, not near the top) — worth knowing before leaning on this as a selling point.
- Adopted: keeping the raw train\_test.csv/validation.csv data files out of the submitted repository. One draft's GITHUB\_SETUP.md flagged that Spotter's assessment data is likely not meant for public redistribution — a legitimate concern the other draft missed. See README.md.
- Changed: weight is represented only by weight\_abs (the corrected value) and a weight\_missing flag. One draft additionally fed the model the raw signed weight; since  $\text{abs}(\text{weight})$  for the negative rows matches the positive-weight distribution almost exactly (Section 3), the sign is a data-entry artifact, not a signal, and keeping it risks the model fitting noise.

- Changed: date features are cyclical only (month\_sin/cos, dow\_sin/cos, plus bounded month/dow). Raw day-of-year and an elapsed-day counter were dropped — both exceed their training range once the model scores November/December dates, which is exactly the kind of extrapolation tree-based models handle poorly.
- Kept from the other draft: two separately-trained models (full vs. reduced feature set) for the two prediction targets, rather than one model fed an all-NaN market\_index/quote\_signal pattern for December it essentially never saw in training.
- Added: an unseen-city holdout experiment (Section 5) to actually measure, rather than just assert, how much the model's accuracy degrades on cities it has never seen — relevant because validation.csv contains 8 such cities.

### 3. Data quality

weight has a sign-flip data-entry error in a subset of rows (values as low as  $-47,500$  lb). This is distinguishable from genuine missingness:  $|\text{weight}|$  for the negative rows matches the distribution of weight for the normal positive rows almost exactly (same mean  $\approx 31,700$  lb, same  $5,000$ – $47,500$  lb range), which is strong evidence of a sign artifact rather than a real, different population. Fix: only the corrected weight\_abs is used as a model feature; the raw signed value is dropped entirely rather than fed alongside it, since it carries no real signal.

Separately, weight and market\_index each have a small fraction of genuinely missing (NaN) values — under 1% in train\_test.csv, up to  $\sim 2\%$  in validation.csv — spread roughly evenly across equipment types, consistent with missing-at-random rather than a systematic collection issue. These are handled with an explicit weight\_missing indicator plus CatBoost's native NaN support, which is a separate mechanism from the weight\_abs fix above; the two are easy to conflate in writing but address different problems (a sign artifact vs. real missingness).

validation.csv contains 8 pickup/delivery cities never seen in train\_test.csv (Chicago, Knoxville, San Diego, Allentown, Jackson, Charlotte, Norfolk, Laredo). This is addressed two ways: geographically, via lat/lon and a haversine geo\_distance feature that generalizes smoothly to new locations; and categorically, via CatBoost's native MISSING-category fallback for pickup/delivery/route values it has never seen. Section 5 measures how well this actually works rather than assuming it does.

### 4. Feature engineering

Categorical (handled natively by CatBoost, no one-hot encoding): pickup, delivery, equipment, route (pickup + delivery concatenated), route\_eq (route + equipment).

Numeric: distance, distance\_log, geo\_distance (haversine distance from lat/lon), distance\_ratio (distance / geo\_distance — captures how road-circuitous a lane is), weight\_abs, weight\_missing, pickup/delivery lat & lon, market\_index, quote\_signal (full model only), month, dow, and cyclical month\_sin/cos, dow\_sin/cos.

Target:  $\log(\text{posted\_rate} / \text{distance})$ , i.e. log rate-per-mile. Predictions are recovered as  $\exp(\text{model output}) \times \text{distance}$ .

### 5. Validation approach

#### 5.1 Time-based vs. random split

The real task is forecasting November/December, dates the model has never seen — so the headline validation number comes from a time-based holdout: train on 2025-01-01 through 2025-09-30, validate on October (never seen during training). A random 85/15 holdout was run on the identical model for comparison only.

| Split                                               | MAE      | RMSE     | Median AE | MAPE  | n     |
|-----------------------------------------------------|----------|----------|-----------|-------|-------|
| Time-based (Jan–Sep train / Oct holdout) — reported | \$114.67 | \$649.41 | \$41.66   | 5.48% | 4,853 |
| Random 85/15 holdout — comparison only              | \$91.26  | \$625.92 | \$26.64   | 3.80% | 7,200 |

Table 1. Same model, same features, two split strategies. The random split is a substantially more optimistic (and less realistic) estimate for this forecasting task.

## 5.2 Unseen-city experiment

To actually measure the risk from Section 3 rather than just flag it: 6 cities (Albany, Bakersfield, Lubbock, New York, Salt Lake City, Syracuse) were removed from training entirely (8,422 rows dropped from the Jan–Sep training set), and the resulting model was evaluated on the October holdout rows that touch one of those 6 cities — simulating exactly what validation.csv does with its 8 real unseen cities — against the same model's performance on ordinary, known-city October rows.

**Cost of a city the model has never seen (Oct holdout, same model)**

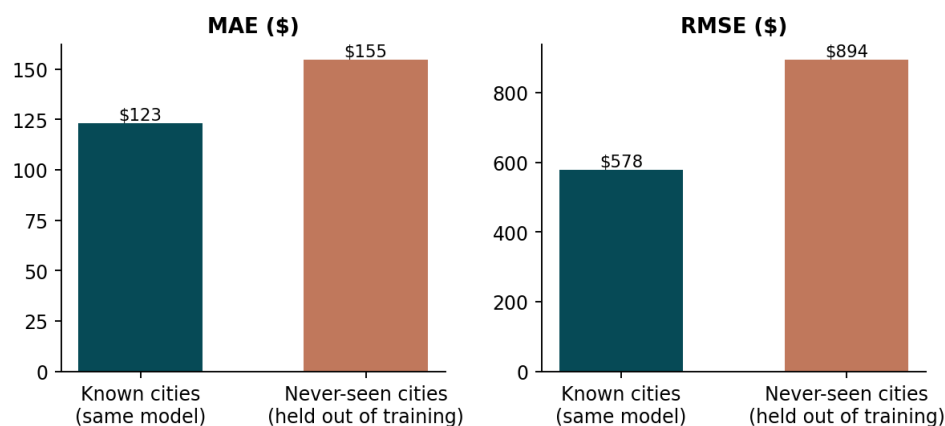


Figure 1. Same model, evaluated on October rows touching a city it never saw in training vs. rows with only known cities.

| Holdout subset            | MAE      | RMSE     | Median AE | MAPE  | n     |
|---------------------------|----------|----------|-----------|-------|-------|
| Known cities (same model) | \$122.95 | \$577.69 | \$55.12   | 6.37% | 3,905 |
| Never-seen-city rows      | \$154.61 | \$893.94 | \$53.25   | 5.82% | 948   |

Table 2. MAE and RMSE are meaningfully worse for never-seen cities (as expected), but median absolute error and MAPE are actually similar or slightly better — the typical prediction isn't much worse, but there are more large misses in the tail. Directionally this confirms the risk from Section 3, but it's a real, moderate degradation rather than a catastrophic one, and the lat/lon + geo\_distance fallback is doing real work.

Caveat: this experiment approximates the real situation but isn't identical to it — the 6 cities chosen here are removed from a Jan–Sep/Oct split of train\_test.csv, while validation.csv's 8 unseen cities appear in Nov/Dec, a period no version of the model has training data for regardless. Treat this as a lower bound on the accuracy gap, not an exact figure.

## 6. Model selection and feature importance

CatBoostRegressor was chosen for the same reasons in both source drafts: native categorical handling (no one-hot matrix for pickup/delivery/route), native missing-value support (no imputation step for weight/market\_index), and it captures nonlinear interactions between distance, equipment, and geography without manual interaction terms. Configuration: depth 7, learning rate 0.06, 200 iterations, L2 regularization 10, seed 42 — selected via the time-based holdout above.

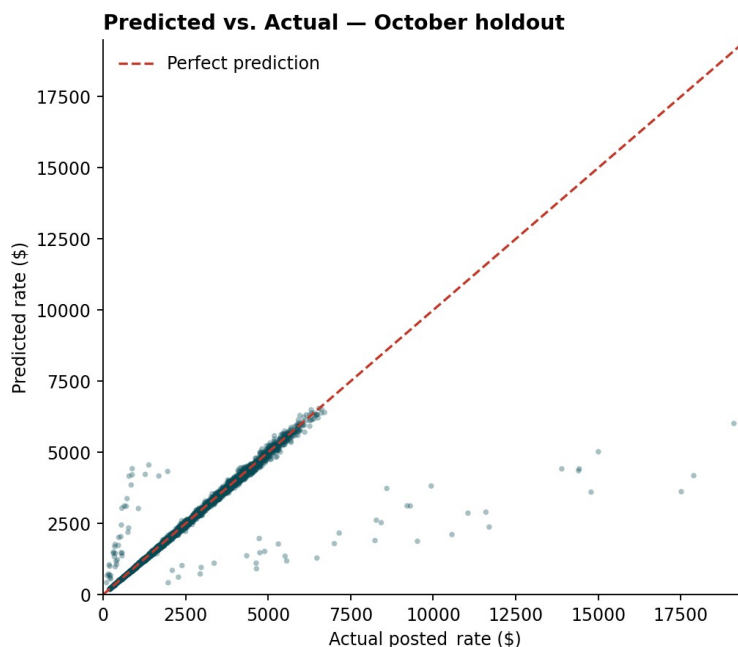


Figure 2. Predicted vs. actual posted\_rate, October holdout. As in both source drafts, a small number of loads carry rate-per-mile values the given features can't explain (visible as the scattered points off the diagonal); these inflate RMSE relative to MAE/median AE.

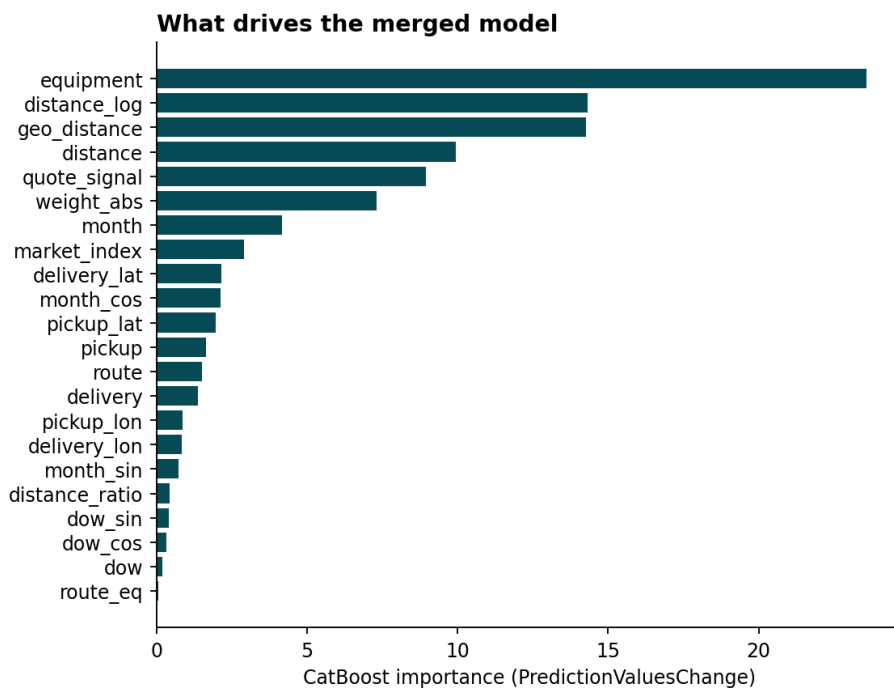


Figure 3. CatBoost feature importance (PredictionValuesChange), full model. Two things worth flagging: (1) distance\_log + geo\_distance + distance + distance\_ratio together ( $\approx 39$ ) outweigh equipment ( $\approx 24$ ) — the “equipment is the top feature” framing in one of the source drafts is an artifact of how importance is split across several correlated distance columns, not evidence that

equipment matters more than distance. (2) route and route\_eq — the lane-identity categoricals adopted from the CatBoost draft on the theory that they'd capture lane-specific pricing — rank near the very bottom (1.51 and 0.05). Once pickup, delivery, and geo\_distance are already in the model, the route string adds little independent signal. This doesn't mean the merge decision to keep route was wrong (it's essentially free, and might still help specific lanes like December's), but it's not the differentiator it looked like on paper.

## 7. Handling the December scenario

december\_chart\_inputs.csv omits lat/lon, market\_index, and quote\_signal. Lat/lon is recovered from a city → coordinate lookup built from train\_test.csv (Lexington and Fort Wayne both appear there). market\_index/quote\_signal cannot be recovered — no December history exists to extrapolate from — so a second model, trained on the same data minus those two columns, is used specifically for December, rather than feeding the full model a pattern (both fields NaN simultaneously) it essentially never saw during training.

| Model                                              | MAE      | RMSE     | Median AE | MAPE  |
|----------------------------------------------------|----------|----------|-----------|-------|
| Full features (→ validation_predictions.csv)       | \$114.67 | \$649.41 | \$41.66   | 5.48% |
| Reduced, no market_index/quote_signal (→ December) | \$129.23 | \$654.41 | \$51.92   | 5.84% |

Table 3. Unlike the near-zero cost seen with a different (HistGradientBoosting) model architecture in an earlier draft of this project, dropping market\_index/quote\_signal costs this CatBoost model a real, measurable ~13% increase in MAE. quote\_signal in particular ranks #5 in Figure 3's importance table — this model leans on it more than the alternative architecture did, so its absence for December is a genuine, not just theoretical, limitation worth stating plainly rather than assuming away.

## 8. December 2025 forecast

The chart below is score.py's output against the completed december\_chart\_inputs.csv. Because pickup, delivery, distance, equipment, and weight are fixed across all 31 rows, only date varies, and the model relies on it through month/dow (and their cyclical encodings). The result is a clean, repeating weekly cycle between about \$811 and \$835 — an honest reflection of what the model learned (a within-week pattern), not evidence of a month-to-month market forecast, which Table 3 shows this model has reduced ability to produce without market\_index/quote\_signal anyway.

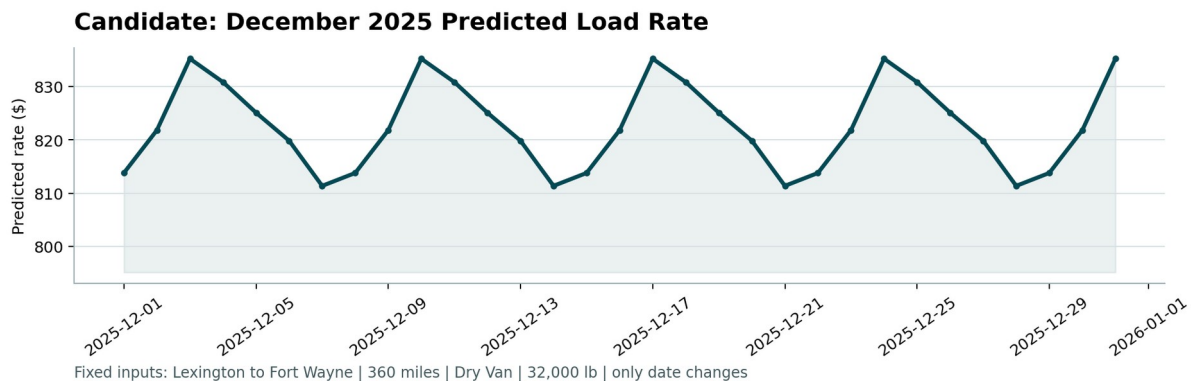


Figure 4. candidate\_december.png, generated by score.py from the submitted december\_chart\_inputs.csv.

## 9. Reproducing this submission

- python -m pip install -r requirements.txt
- python src/train.py — cleans the data, runs Sections 5–7's comparisons, saves the two final models to models/
- python src/predict.py — writes validation\_predictions.csv and the completed data/december\_chart\_inputs.csv

- `python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv`

Note: `train_test.csv` and `validation.csv` are not included in the submitted GitHub repository (see `README.md`) — place them in `data/` locally before running. This avoids redistributing Spotter's assessment data in a public repo.